Table of Contents

Week 5 — Summary & Examples Reference

Alternate Real-World Examples for Every Part of the Lesson

Use alongside: Week5_Constructors.md | **Class:** Wednesday, 1 July 2026 · 6:30 PM

> *This file mirrors the main lesson guide part-by-part, but every example here is **different** from the main file. Use these when a student needs a second angle, or to add variety mid-discussion without repeating yourself.*

One-Line Definitions (Read These Aloud)

> ***A constructor** is a one-time setup action that runs automatically the moment an object is created, ensuring the object is complete and valid before anything else can use it.*

> ***A default value** is a meaningful fallback used when a caller does not provide specific information — it must be logically valid, not just "nothing."*

> ***Constructor overloading** means offering multiple setup paths — full details, partial details, or minimal — all of which produce the same type of complete, valid object.*

> ***The master constructor** is the single setup path where all validation and attribute-setting actually happens. Every other path hands off to it.*

> ***Chaining** is when a convenience constructor delegates to the master constructor as its very first action.*

Part 1 Alternate Examples — The Problem: Incomplete Objects

> *Main file used: hotel check-in (key before the room is ready).*
> ***This file uses:** a patient registration form and a job application.*

Alternate Story — Incomplete Hospital Patient Registration

> *"A new patient arrives at a hospital. The nurse creates a patient record in the system, enters the name, then gets called away for an emergency. She never comes back to finish entering the date of birth, blood type, or allergy information.*
>
> *The patient record now exists in the hospital system — incomplete, with critical missing fields. Two days later, a doctor sees this patient in an emergency and checks the system. It shows no blood type, no allergy history."*

```
THE THREE PROBLEMS — APPLIED TO HOSPITAL REGISTRATION
──────────────────────────────────────────────────────────────

PROBLEM 1 — INCOMPLETE OBJECTS CAN EXIST
  A Patient record with no blood type or allergy info is
  dangerous — but the system holds it anyway, without complaint.

PROBLEM 2 — NO ENFORCED ORDER
  Should allergies be recorded before blood type, or after?
  The system doesn't enforce it — a nurse might enter them
  in any order and miss the dependencies between fields.

PROBLEM 3 — REPEATED SETUP SCATTERED EVERYWHERE
  Every ward, every clinic, every nurse station creates
  Patient records — all repeating the same 5-step setup.
  A new mandatory field added to Patient records breaks
  every single creation point across the entire hospital.
──────────────────────────────────────────────────────────────
```

Alternate Story — Half-Submitted Job Application

> *"A job seeker starts filling in an online application — name, contact email — and then closes the browser tab halfway. The company's system has saved a 'partial application' with no qualifications, no experience, no references.*
>
> *The hiring manager sees this in the system and cannot do anything useful with it. The candidate won't be contacted because the email was entered but no position was selected. This half-created 'object' is useless noise in the system."*

Part 2 Alternate Examples — What Is a Constructor?

> *Main file used: hotel receptionist collecting all info before handing over a room key.*
> ***This file uses:*** *a café custom drink order and a passport application office.*

Alternate Analogy — Ordering a Custom Drink at a Café

> *"When you order a custom drink at a café, the barista does NOT hand you a cup immediately and then ask what you want one ingredient at a time over the next ten minutes. They ask everything upfront:*
> *'What size? Hot or iced? What kind of milk? Any syrup? Extra shot?'*
>
> *Only after all of this is settled do they start making the drink — and only when it's fully made do they hand it to you. You receive a* ***complete*** *drink, not a partial one."*

```
CAFÉ ORDER — A CONSTRUCTOR IN REAL LIFE
──────────────────────────────────────────────────────────

WHAT THE BARISTA (constructor) DOES:
  → Collects all required information upfront (parameters):
    size, temperature, milk type, extras
  → Checks each piece of information is valid (validation):
    "We don't have oat milk today — shall I suggest almond?"
  → Makes the drink using that information
  → ONLY THEN hands over the finished drink (object is ready)

THE RESULT:
  → The moment you receive the cup (the object), it is complete
  → You never receive a half-made drink and finish it yourself
  → If you ordered incorrectly, you are corrected before it's made
──────────────────────────────────────────────────────────
```

Alternate Analogy — Passport Application Office

```
PASSPORT APPLICATION
──────────────────────────────────────────────────────────

PROPERTY 1 — RUNS AUTOMATICALLY ON SUBMISSION
  The moment you submit the form and fee, the passport process
  starts on its own. You don't call the office daily to trigger
  each step — the setup happens automatically.

PROPERTY 2 — RUNS EXACTLY ONCE
  A passport is created once. You don't go through the full
  application process each time you use your passport.
```

```
PROPERTY 3 — NOT ISSUED UNTIL COMPLETE
  You do not receive a passport until all checks are done —
  photo verified, identity confirmed, fee paid, printing finished.
  The passport is either fully ready or not issued at all.
  No half-issued passport exists.
──────────────────────────────────────────────────────────────
```

Part 3 Alternate Examples — Flexible Constructors: Defaults and Overloading

> *Main file used: hotel walk-in vs full booking (defaults for standard room, 1 night).*
> ***This file uses:*** *a pharmacy prescription system and a school library card.*

Alternate Analogy — Pharmacy Prescription System

> *"A pharmacy creates a prescription record each time a doctor sends one. The doctor's system sends different amounts of information depending on the situation."*

```
PHARMACY PRESCRIPTION — THREE SETUP PATHS
──────────────────────────────────────────────────────────────────────

PATH A — Full information provided
  Doctor sends: patient name, medication name, dosage, duration, refills
  → All five fields set from what was given

PATH B — Standard prescription (no refills mentioned)
  Doctor sends: patient name, medication name, dosage, duration
  → refills defaults to 0 (a sensible, valid default for most medications)

PATH C — Emergency repeat prescription (known patient, same medication)
  Pharmacist enters: patient name only
  → medication = last prescribed, dosage = last dosage, duration = 7 days
    (all defaults based on the patient's history)

ALL THREE PATHS produce a valid, usable prescription record.
The pharmacist chooses the path that matches what they know.
──────────────────────────────────────────────────────────────────────
```

Alternate Analogy — School Library Card

```
LIBRARY CARD — DEFAULTS MAKE SENSE
──────────────────────────────────────────────────────────────────────
```

```
When a new student gets a library card, not all information is
always available on the first day:

  name — always required (no card without a name)
  studentId — always required
  membershipType — defaults to "Standard" if not specified
  booksAllowed — depends on membershipType: Standard = 5, Premium = 10

A student who doesn't specify their membership type gets
"Standard" with 5 books allowed — a meaningful, valid default.
NOT "null books allowed" or "-1 books" — those would be invalid.

A good default is a VALID fallback, not just "nothing."
──────────────────────────────────────────────────────────────────
```

Part 4 Alternate Examples — The Master Constructor Principle

> *Main file used: hotel receptionist and the master guest record form.*
> ***This file uses:*** *a university student enrolment office and a customs form at an airport.*

Alternate Analogy — University Student Enrolment

> *"A university has three registration paths: in-person at the office, online self-service, and through a faculty coordinator. All three look different to the student. But in the background, every single registration — no matter how it arrived — goes through the same Enrolment Office process:*
>
> *Validate name → Check student ID → Confirm major → Assign student number → Issue card.*
>
> *That Enrolment Office process is the master constructor. The three paths are just different front doors to the same core process."*

```
WITHOUT MASTER PROCESS                  WITH MASTER PROCESS
──────────────────────────────────      ──────────────────────────────────
In-person: validates name,              In-person: "hand off to
  checks ID, assigns number               Enrolment Office"
Online: validates name, checks          Online: "hand off to
  ID, assigns number                      Enrolment Office"
Faculty coordinator: validates          Faculty coordinator: "hand off
  name, checks ID, assigns number        to Enrolment Office"

3 places to fix a validation bug        1 place to fix a validation bug
3 places to add a new required step     1 place to add a new required step
```

```
Easy to forget one                    Impossible to miss
_____________________________         _____________________________
```

Alternate Analogy — Customs Declaration Form at an Airport

```
CUSTOMS ARRIVAL CARDS — CHAINING IN PRACTICE
_____________________________________________________________________

Some airports have simplified forms for frequent travellers —
they only need to fill in name and passport number. The border
officer fills in the rest using passport data on record.

Some have full-form arrivals for first-time visitors — every
field must be filled in by the traveller.

Both paths END at the same customs officer's terminal, where
ALL the same checks are run:
  → Name matches passport?
  → Purpose of visit valid?
  → Declaration complete?

The customs officer's terminal is the MASTER.
The two form types are convenience paths that hand off to it.

Critical rule: the handoff to the customs officer must happen
FIRST — before the traveller is admitted. Nothing is done
before the master process completes.
_____________________________________________________________________
```

Part 5 Alternate Example — Java as Illustration

> *Main file showed the Student class.*
> ***This file shows the same concept using a LibraryMember class.***

```java
// LibraryMember — same constructor concept, different domain

public class LibraryMember {

    private String name;
    private String memberId;
    private String membershipType;
    private int    booksAllowed;

    // MASTER CONSTRUCTOR — all validation and setup here
```

```
    public LibraryMember(String name, String memberId, String membershipType) {
        setName(name);        setMemberId(memberId);
        setMembershipType(membershipType);   // also sets booksAllowed via rule    }
            // CONVENIENCE CONSTRUCTOR – standard membership assumed
    public LibraryMember(String name, String memberId) {
        this(name, memberId, "Standard");    // hands off to master with default    }
            // CONVENIENCE CONSTRUCTOR – minimal: name only, system assigns the rest
    public LibraryMember(String name) {
        this(name, "AUTO-" + System.currentTimeMillis(), "Standard");    }
    // Validated setters (encapsulation from Week 4)    private void setName(String name) {
        this.name = (name ≠ null && !name.isBlank()) ? name.trim() : "Unknown";    }
            private void setMemberId(String id) {
        this.memberId = (id ≠ null && !id.isBlank()) ? id : "TEMP-000";    }
    private void setMembershipType(String type) {        if ("Premium".equals(type)) {
            this.membershipType = "Premium";            this.booksAllowed   = 10;
        } else {            this.membershipType = "Standard";
            this.booksAllowed   = 5;        }    }                public String toString() {
        return "Member{" + name + ", " + memberId
             + ", " + membershipType + ", " + booksAllowed + " books}";    }    }
```

```
// THREE CREATION PATHS – all produce valid, complete objects
LibraryMember m1 = new LibraryMember("Sokha", "LIB-001", "Premium");
LibraryMember m2 = new LibraryMember("Dara",  "LIB-002");         // Standard default
LibraryMember m3 = new LibraryMember("Chenda");                   // All defaults

System.out.println(m1);  // Member{Sokha, LIB-001, Premium, 10 books}
System.out.println(m2);  // Member{Dara,  LIB-002, Standard, 5 books}
System.out.println(m3);  // Member{Chenda, AUTO-xxx, Standard, 5 books}
```

Concept Translation Table

Java code	OOP concept meaning
`public LibraryMember( ... )`	"This is the setup action for a member"
`this(name, memberId, "Standard")`	"Hand off to master with a default"
`setMembershipType(type)`	"Validate the type, then set books limit"
`new LibraryMember("Sokha","P")`	"Build a complete member using setup"

Incomplete Object Scenarios — Use in Class Discussion

Scenario 1 — The Café Order Problem

```
Object: DrinkOrder
Setup without constructor:
  Step 1: Create empty DrinkOrder         → order exists, all fields empty
  Step 2: setSize("Large")
  Step 3: setType("Iced Americano")
  Step 4: setSugar("None")
  → Barista calls prepare() after Step 2 by accident
  → They make a Large drink of unknown type with unknown sugar

Question: Which problem from Part 1 applies here?
Answer:   Problem 1 — an incomplete object existed between
          steps and was used before setup was complete.

How would a constructor fix this?
  → The order is not created until all required fields are given.
  → The barista cannot call prepare() on an incomplete order.
```

Scenario 2 — Defaults vs Invalid Values

```
Object: LibraryMember
Someone creates a member and doesn't provide a membershipType.

Option A — default = null
  booksAllowed = ??? (system crashes when checked)
  → This is NOT a valid default.

Option B — default = "Standard" (booksAllowed = 5)
  booksAllowed = 5
  → The object is in a valid, usable state immediately.
  → The default is MEANINGFUL — it represents a real, logical choice.

Question: Why is "null" not a valid default for membershipType?
Answer:   Because it leaves the object in an unusable state —
          the very problem constructors exist to prevent.
```

Scenario 3 — The Handoff Must Be First

```
Object: LibraryMember convenience constructor
Situation: A developer tries to do some logging BEFORE handing off
           to the master constructor.

WRONG approach (in code, this would be a compile error):
```

```
System.out.println("Creating member: " + name);   ← does other things first
      this(name, "AUTO-001", "Standard");               ← then hands off
  }

WHY IT'S WRONG:
  The master hasn't run yet — no attributes are set.
  Logging "Creating member: " + name while the object has
  no validated data yet is accessing an incomplete object.

RIGHT approach:
  LibraryMember(String name) {
      this(name, "AUTO-001", "Standard");   ← hands off FIRST
      // any extra work AFTER the master is done
  }
```

Student Discussion Questions

1. "From your own experience — think of a process you go through (applying for something, ordering something, registering for something) where you must provide ALL the required information before the process accepts your request. How is that like a constructor?"
2. "Why is `null` or `0` not always a valid default? Give a real-world example where using 0 as a default would cause a real problem."
3. "If each setup path (convenience constructor) did its own validation separately, and a new rule was added to the system, what would a developer need to do? What happens if they miss one path?"
4. "The customs form analogy: why must the handoff to the customs officer happen BEFORE the traveller enters the country — not after? Connect this to why the master constructor must run before any other work."
5. "A constructor cannot be called again after an object is created. Why is this actually a feature, not a limitation? What would go wrong if you could re-run a constructor on an existing object?"

Quick-Reference: All Four Weeks Together

```
WEEKS 2–5 — HOW EACH WEEK'S CONCEPT CONNECTS
──────────────────────────────────────────────────────────────

WEEK 2: "What is an object?"
  An object holds real data and can do things.

WEEK 3: "What does an object hold?"
  Attributes (persistent data), methods (actions), state.
```

```
WEEK 4: "How is an object's data protected?"
  Private attributes, public getters, setters with validation.

WEEK 5: "How does an object start life correctly?"
  A constructor collects all required information upfront,
  validates it, and sets everything before the object is released.

Together: a properly designed object is born valid (constructor),
stays valid (encapsulation + validation), and behaves correctly
(methods that check before changing state).
──────────────────────────────────────────────────────────────────────
```

Common Student Confusions — Quick Fixes

If a student says...	Respond with...
"I can just call setters right after creating the object — same thing"	"Like the café — you wouldn't want a half-made drink handed to you mid-preparation. A constructor ensures the object is NEVER in a half-made state."
"What's wrong with a null default?"	"A null membershipType means booksAllowed cannot be calculated. The object is immediately broken. A default must be meaningful — 'Standard' is a real, valid choice."
"Why can't I do other things before handing off to the master?"	"Like customs — you must clear the checkpoint before entering the country, not after. The master sets up the object. Nothing should run on an uninitialized object."
"All this chaining seems complicated — why not just copy the validation into each path?"	"Same reason the university uses one Enrolment Office for all three registration paths. One place to fix. One place to update. Change it once, all paths benefit."

Lesson Arc Summary (Mapped to Both Example Sets)

```
HOW THE TWO EXAMPLE SETS LINE UP, PART BY PART
──────────────────────────────────────────────────────────────────────
PART                    MAIN FILE EXAMPLE       THIS FILE'S EXAMPLE
```

```
──────────────────────────────────────────────────────────────────────
Part 1 — The Problem    Hotel key before        Patient record with
                         room is ready           missing critical fields
                                                 + half-submitted job form

Part 2 — Constructor    Hotel receptionist:     Café custom drink:
                         all info upfront,       all ingredients upfront,
                         then key handed over    then drink handed over
                                                + Passport: not issued
                                                  until fully processed

Part 3 — Defaults &     Hotel packages:         Pharmacy prescription:
Overloading              full booking,           full / standard / repeat
                         walk-in, loyalty card  + Library card with
                                                  defaults for type/limit

Part 4 — Master         Hotel master guest      University Enrolment Office
Constructor              record form             (all 3 paths → same office)
                                                 + Customs form (2 forms →
                                                   same officer's terminal)

Part 5 — Java           Student class           LibraryMember class
Illustration             (name, age, major)      (name, memberId, type)
──────────────────────────────────────────────────────────────────────
Same concepts. Different stories. Use whichever lands best.
```

Homework Reminder

> ***No Java code required this week (optional bonus only).***

Design three `Smartphone` setup paths in plain English:
1. Full path: brand + model + storageGB
2. Partial path: brand + model → storageGB defaults to 64
3. Minimal path: brand only → model = “Standard”, storageGB = 32

Identify the master path. Write 3–5 sentences explaining why one master path is better than three separate validations.

Week 5 Summary Reference | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com